

State Vector Simulation of Distributed Krylov Quantum Diagonalization

Aadithya Iyer SR No: 25828
Department of Computational and Data Sciences
Indian Institute of Science, Bangalore
aadithyaiyer@iisc.ac.in

Abstract—The time-independent Schrödinger equation $\hat{H}\psi = E\psi$ governs many-body quantum systems and has several uses in quantum chemistry, condensed matter, and high-energy physics. Representing the Hamiltonian $\hat{H}$ for a system consisting of just 20 atoms is a $2^{20} \times 2^{20}$ matrix, making exact diagonalization infeasible. Krylov Quantum Diagonalization (KQD) [1] builds a compact Krylov subspace $\mathcal{K}^r = \text{span}\{|v\rangle, A|v\rangle, A^2|v\rangle, \dots, A^{r-1}|v\rangle\}$ via time-evolved quantum states $|e^{-iHkt}\psi_0\rangle$, extracts matrix elements via the Efficient Hadamard Test [2], and solves the formed generalised eigenvalue problem $\tilde{H}c = E\tilde{S}c$ classically. In this work, a distributed, GPU-accelerated statevector simulator for KQD on the Heisenberg spin chain (representation of the Hamiltonian matrix), comparing CUDA-Q, CUDA/MPI, and OpenMP/MPI backends is presented. The key contributions are as follows: the implementation of the KQD algorithm on the CUDAQ quantum simulator, a handwritten CUDA kernel suite implementing single-qubit, fused 4×4 two-qubit (RXX/RYY/RZZ), and shared-memory tree-reduction kernels with direct bit-arithmetic indexing; MPI task parallelism distributing independent Krylov vectors across 3 GPUs with statevector reuse to eliminate the $4 \times$ circuit repetition required by the IBM circuit-based reference; and compile-time thread coarsening strategies. A speedup of $6.2 \times$ over the CUDAQ baseline at 23 qubits. Krylov convergence was also validated against exact diagonalization for proof of code correctness. Nsight Systems profiling identifies the expectation-value reduction as the dominant kernel, motivating its shared-memory tree-reduction design.

I. INTRODUCTION

Many fundamental problems in condensed matter physics, quantum chemistry, and high-energy physics reduce to finding the ground-state energy $\lambda_{\min}$ of a Hamiltonian $\hat{H}$ via $\hat{H}\psi(x) = E\psi(x)$. Classical Krylov methods project $\hat{H}$ onto $\mathcal{K}^r = \text{span}\{|v\rangle, A|v\rangle, \dots, A^{r-1}|v\rangle\}$, but the Hamiltonian of 20 atoms is by construction a $2^{20} \times 2^{20}$ matrix—exponentially scaling the complexity of our molecule makes classical solvers infeasible.

Theorem [1]: The Krylov subspace generated by H on $|\psi_0\rangle$ equals the subspace spanned by $\mathcal{K}_U^r = \text{span}\{|\psi\rangle, U|\psi\rangle, \dots, U^{r-1}|\psi\rangle\}$, where $U = e^{-iHt}$, enabling quantum Krylov construction and classical GEVP solve [3].

Research Gaps & Contributions: The work as per the IBM documentation uses the Qiskit [4] Python library to simulate their quantum circuits. A pre-existing implementation of the quantum algorithm on Nvidia’s CUDA-Q platform [5], which is built on a C++ backend, does not exist. The first deliverable is the implementation of the algorithm on CUDA-Q, taking the math presented in the IBM documentation as reference. This

serves as a good baseline for comparing further optimisations to the algorithm. A GPU statevector simulator is subsequently developed using custom CUDA kernels for quantum gate applications. Lastly, a structurally identical CPU version is also developed using OpenMP/MPI. Further MPI-level parallelism is exploited by distributing the Krylov subspace construction across three ranks, reflecting multi-QPU deployment (IBM Kingston, Fez, Marrakesh).

Understanding how gates act on a statevector is central to this work. A quantum gate is a unitary operation that transforms the amplitudes of a statevector—analogueous to a matrix-vector multiply on the 2^N complex amplitudes that represent the state of a system [6]. Applying a quantum gate to qubit q mixes only the amplitude pairs that differ at bit position q , leaving all others unchanged. This locality makes statevector simulation embarrassingly parallel. Each pair can be updated independently and simultaneously.

II. RELATED WORK & BACKGROUND

The quantum subspace diagonalization framework presented in [1] demonstrated convergence guarantees for the Generalized Eigenvalue Problem (GEVP) method and identified the Toeplitz structure of the overlap and Hamiltonian matrices. Babbush et al. [3] demonstrated hardware on Google’s superconducting processor, achieving ground-state estimation for classically intractable many-body Hamiltonians, thereby establishing KQD as a viable near-term quantum algorithm.

The IBM Quantum Learning course on KQD [2] is the direct algorithmic basis for this project. It introduces the *Efficient Hadamard Test*—a significant improvement over the standard Hadamard Test. The circuit is restructured so that $\text{Re}[S(k)]$ and $\text{Im}[S(k)]$ are obtained from $\langle X_{\text{anc}} \rangle$ and $\langle Y_{\text{anc}} \rangle$ via single-qubit measurements after applying Hadamard or $S^\dagger$ +Hadamard to the ancilla. This eliminates controlled- U operations, substantially reduces circuit depth, and enables straightforward statevector simulation.

The qHipSTER framework [7] pioneered statevector simulation on distributed memory, introducing the paired-amplitude (single-qubit) and quartet-amplitude (two-qubit) kernel patterns. The essential observation is that a gate on qubit q necessitates just pairs of amplitudes that vary in bit q , which can be calculated using modulo and floor operations, hence facilitating highly parallel execution. NVIDIA CUDA-Q [5] and cuStateVec [8] offer high-level GPU simulation abstractions

utilized as benchmarks for accuracy and performance; our lower-level CUDA kernels reveal coarsening, kernel fusion, and MPI parallelism not accessible via CUDA-Q.

The IBM Quantum Learning course [2] offers a comprehensive Python/Qiskit implementation of KQD, encompassing a mathematical description of the algorithm, circuit building, Trotterization, and expectation-value extraction. Our research utilizes this as the mathematical reference and re-implements the complete pipeline in C++. Initially, NVIDIA CUDAQ [5] (a C++ framework that provides high-level quantum circuit primitives and facilitates GPU-accelerated quantum circuit simulation) is employed. Subsequently, manually crafted CUDA kernels and OpenMP functions are incorporated, directly manipulating the statevector array, circumventing the CUDA-Q abstraction to reveal parallelism at the kernel level.

III. METHODOLOGY

A. Statevector Representation and KQD Setup

An n_Q -qubit state (represented as $|\psi\rangle = \sum_i \alpha_i |i\rangle$) is stored as a flat double2 array of size 2^{n_Q} on the device. Due to the nature of evolving our initial state via Trotterisation, the Krylov matrices are Toeplitz, so only the first row of our projected hamiltonian and overlap matrix are needed: $S(k) = \langle \psi_0 | U^k | \psi_0 \rangle$, $H(k) = \langle \psi_0 | H U^k | \psi_0 \rangle$. The time step is dt taken from [2], with:

$$dt = \frac{\pi}{\|H\|}, \quad \|H\| = \max(|\lambda_{\min}|, |\lambda_{\max}|). \quad (1)$$

B. Distributed KQD: MPI Task Parallelism Across 3 GPUs

Each Krylov vector k uses only $|\psi_0\rangle$ and produces a single complex pair $(S(k), H(k))$. This is exploited with three-way MPI task parallelism. The $k=0$ element is analytical; the remaining $k \in \{1, 2, 3\}$ are distributed across three ranks. Each rank pins itself to a distinct GPU via `cudaSetDevice(rank % num_gpus)`, runs the entire $\text{PREP} \rightarrow U(t) \rightarrow \text{UNPREP}$ pipeline on its assigned k (see Fig 1), and contributes four doubles (Re/Im of S and H) to a single `MPI_Gather` on rank 0. Inter-rank communication during computation is therefore zero; the only synchronisation is one `MPI_Barrier` for timing and one final gather of 12 doubles. This pipeline is illustrated in Fig. 1.

Key optimisation — evolve once and branch copies. For each k , the IBM reference runs four full $\text{PREP} + U(t)$ circuits to obtain Re/Im of $S(k)$ and $H(k)$. In our simulator the Trotter evolution is the most expensive stage, but its output state is fully reusable. Therefore, the helper `run_trotter()` is executed *once* per rank, and `cudaMemcpyDeviceToDevice` the resulting 2^{n_Q+1} amplitudes into separate buffers (`sv_s`, `sv_s_im`, `sv_re`, `sv_im`). This eliminates 3 of every 4 Trotter sweeps and is the primary reason our OMP CPU statevector backend keeps pace with CUDA-Q on small problems (Fig. 3).

C. CUDA Kernel Engineering — Single-Qubit Gates

Applying gate $Q \in \mathbb{C}^{2 \times 2}$ to qubit q updates 2^{n-1} disjoint amplitude *pairs* $(\alpha_{i_0}, \alpha_{i_1})$ that differ only at bit q . The challenge is to map each CUDA thread directly to its pair without

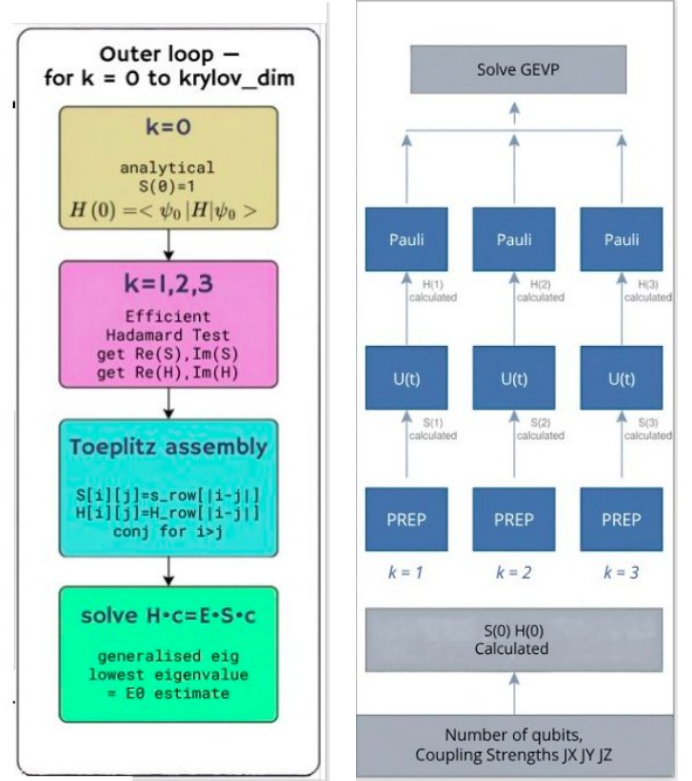


Fig. 1: Krylov Quantum Diagonalization workflow (Unpublished Diagram)

locks, atomics, or branch divergence. To do this, qHiPSTER's paired-amplitude arithmetic [7] is adopted: $lo = tid \bmod 2^q$, $hi = \lfloor tid/2^q \rfloor$, $i_0 = hi \cdot 2^{q+1} + lo$, $i_1 = i_0 + 2^q$. Each thread reads $(\alpha_{i_0}, \alpha_{i_1})$, multiplies by Q , and writes the result back; no warp-level synchronisation is needed because every thread touches a disjoint pair.

Eight such kernels were coded from scratch (`apply_x_gate`, `_y`, `_z`, `_hadamard`, `_s`, `_sdg`, `_cx`, `_rz`). Each launches with $B=256$ threads/block; the launch configuration is $\lceil 2^{n-1}/(B \cdot C) \rceil$ blocks where the **compile-time coarsening factor** $C \in \{1, 2, 4\}$ controls how many pairs each thread handles sequentially. $C=2$ gives the best throughput-per-watt by raising arithmetic intensity without over-serialising threads (§IV).

D. CUDA Kernel Engineering — Fused Two-Qubit Gates

Two-qubit gates couple groups of four amplitudes $(i_{00}, i_{01}, i_{10}, i_{11})$ spanning all $\{0, 1\}$ combinations at qubit positions $lo = \min(q_1, q_2)$ and $hi = \max(q_1, q_2)$. Extending the bit arithmetic requires a *three-segment decomposition* of tid : $inner = tid \bmod 2^{lo}$, $mid = \lfloor tid/2^{lo} \rfloor \bmod 2^{hi-lo-1}$, $outer = \lfloor tid/2^{hi-1} \rfloor$, giving:

$$i_{00} = outer \cdot 2^{hi+1} + mid \cdot 2^{lo+1} + inner$$

$$i_{01} = i_{00} + 2^{lo}, \quad i_{10} = i_{00} + 2^{hi}, \quad i_{11} = i_{00} + 2^{lo} + 2^{hi}$$

Fused 4×4 kernels. Rather than decomposing $R_{XX}/R_{YY}/R_{ZZ}$ into single-qubit gates plus CNOTs (which the CUDA-Q baseline does—requiring ~ 10 kernel

launches per Trotter bond), each rotation is implemented as a single in-place 4×4 kernel. For $R_{XX}(\theta)$ with $c = \cos(\theta/2)$, $s = \sin(\theta/2)$, $\theta = 2J_X \delta t$:

$$R_{XX} = \begin{pmatrix} c & 0 & 0 & -is \\ 0 & c & -is & 0 \\ 0 & -is & c & 0 \\ -is & 0 & 0 & c \end{pmatrix}.$$

The kernel reads four amplitudes, computes four updates, and writes them back; one launch per gate, three launches per Trotter bond ($R_{XX} \rightarrow R_{YY} \rightarrow R_{ZZ}$). R_{YY} uses the same quartet structure with sign flips at i_{00}/i_{11} ; R_{ZZ} is purely diagonal (multiply same-parity amplitudes by $e^{-i\theta/2}$, opposite-parity by $e^{+i\theta/2}$).

Second-order Suzuki-Trotter. [9] Even Trotter steps apply even-then-odd bonds in $R_{XX} \rightarrow R_{YY} \rightarrow R_{ZZ}$ order; odd steps reverse *both* bond order and gate sequence ($R_{ZZ} \rightarrow R_{YY} \rightarrow R_{XX}$), cancelling $O(\delta t^2)$ errors at no extra kernel cost.

Ancilla-sector extraction. The Trotter evolution acts only on the ancilla- $|1\rangle$ sector (odd-indexed amplitudes). Two helper kernels (`extract_odd_sector`, `insert_odd_sector`) compact the 2^{n_Q+1} -amplitude full vector into a 2^{n_Q} system buffer before Trotter, then re-insert afterwards. This halves DRAM traffic during the most kernel-dense phase of the algorithm.

E. CUDA Reduction Kernel for $\langle X_{\text{anc}} \rangle$

The expectation value $\langle X_{\text{anc}} \rangle = P(\text{anc}=0) - P(\text{anc}=1) = \sum_i (-1)^{i \bmod 2} |\alpha_i|^2$ collapses 2^{n_Q+1} complex amplitudes into a single scalar. This is implemented as a two-stage reduction (Listing 1): each thread strides over the statevector accumulating a private `val`; a 256-element `__shared__` array `sdata` performs an in-block halving tree reduction ($128 \rightarrow 64 \rightarrow \dots \rightarrow 1$ in $\log_2 256 = 8$ steps); thread 0 of each block then issues a single `atomicAdd` to a global double. Without shared memory, all 256 threads per block would contend for the same global atomic, serialising at the memory controller; with shared memory, only one atomic is issued per block.

Listing 1: Shared-memory tree reduction for $\langle X_{\text{anc}} \rangle$.

```
__global__ void reduce_x_expectation(const double2 *sv,
                                   long long ns, double *out) {
    __shared__ double sdata[256];
    long long tid = blockIdx.x * blockDim.x + threadIdx.x;
    double val = 0.0;
    for (long long i = tid; i < ns; i += blockDim.x * gridDim.x) {
        double prob = sv[i].x * sv[i].x + sv[i].y * sv[i].y;
        val += (i % 2 == 0) ? prob : -prob;
    }
    sdata[threadIdx.x] = val; __syncthreads();
    for (int s = blockDim.x/2; s > 0; s >>= 1) {
        if (threadIdx.x < s) sdata[threadIdx.x] += sdata[threadIdx.x+s];
        __syncthreads();
    }
    if (threadIdx.x == 0) atomicAdd(out, sdata[0]);
}
```

F. OpenMP Mirror Implementation

The CPU port mirrors every CUDA kernel structurally—but the index arithmetic is bit-for-bit identical—but the engineering decisions differ. Three choices were non-trivial:

(1) Schedule policy. Gate functions are parallelised via `#pragma omp parallel for schedule()`, wherein scheduling is set to either static or dynamic and the timings of both cases are calculated separately. Similarly, the reduction

kernel is set to use both static as well as dynamic scheduling. It is observed that contiguous chunks per thread enable sequential DRAM access and hardware prefetching. Using dynamic for the reduction could be the dominant source of overhead in the OMP Dynamic backend (§IV).

(2) Reduction clause. The expectation value uses a single OpenMP directive (Listing 2); each thread receives a private accumulator and OpenMP merges them at the parallel-for barrier. This is the CPU equivalent of the GPU’s shared-memory tree plus `atomicAdd`, expressed in one line.

Listing 2: Mirroring GPU shared-memory reduction.

```
double measure_x(const double2 *sv, long long ns) {
    double val = 0.0;
    #pragma omp parallel for reduction(+:val) schedule(static)
    for (long long i = 0; i < ns; i++) {
        double prob = sv[i].x * sv[i].x + sv[i].y * sv[i].y;
        val += (i % 2 == 0) ? prob : -prob;
    }
    return val;
}
```

(3) Memory model. CUDA kernels operate on raw `cudaMalloc’d` device pointers; OpenMP gate functions take a `std::vector<double2>` shared across threads. The `run_trotter()` helper returns the vector by value, which the C++ move semantics turn into a pointer swap (zero copy), letting the same evolved state be shared across all four measurement branches—the CPU equivalent of `cudaMemcpyDeviceToDevice`.

IV. EXPERIMENTS AND RESULTS

Setup: $J_X = J_Y = J_Z = 0.5$, $K = 4$, Trotter = 4, dt from Eq. (1). GPU: NVIDIA RTX A5000 $\times 3$ with MPI. CPU: 32 OpenMP threads/rank. Wall time excludes GEVP solve.

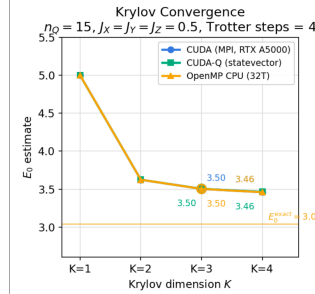


Fig. 2: Krylov convergence ($n_Q = 15$, $J_X = J_Y = J_Z = 0.5$, Trotter steps = 4).

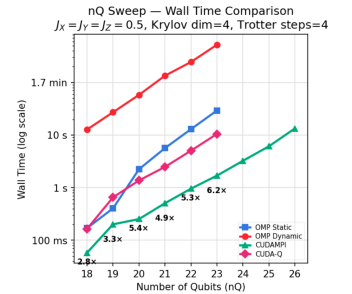


Fig. 3: Wall time vs. n_Q (18–26).

A. Krylov Convergence. Figure 2 confirms correctness: all three backends (CUDA/MPI, CUDA-Q, OpenMP 32T) produce *identical* E_0 estimates at every K . The estimate falls from 5.0 ($K=1$) through 3.63, 3.50, to 3.46 ($K=4$), converging toward $E_0^{\text{exact}} \approx 3.04$. This validates that our hand-written statevector kernels exactly reproduce the CUDA-Q reference output.

B. Qubit Scaling (n_Q Sweep). Figure 3 plots wall time vs. n_Q on a log scale. CUDA/MPI achieves $2.8\times$ at $n_Q=18$, growing to $6.2\times$ at $n_Q=23$ over CUDAQ, reflecting GPU parallelism combined with the statevector-reuse optimisation eliminating $3\times$ Trotter sweeps per Krylov vector. OMP Dynamic is the *slowest* backend: dynamic scheduling causes out-of-order iteration dispatch, producing non-contiguous DRAM fetches and cache-line thrashing across the 2^{n_Q} statevector.

OMP Static and CUDA-Q perform similarly at small n_Q ; CUDA/MPI overtakes beyond $n_Q \approx 20$.

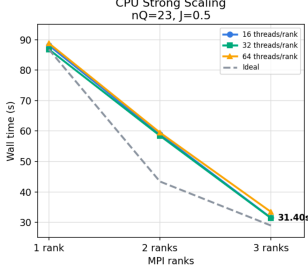


Fig. 4: CPU strong scaling ($n_Q = 23$, $J = 0.5$).

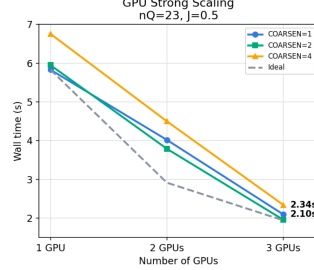


Fig. 5: GPU strong scaling ($n_Q = 23$, $J = 0.5$).

C. CPU Strong Scaling. Figure 4 shows wall time vs. MPI ranks at $n_Q = 23$. Scaling is near-ideal: ~ 88 s (1 rank) $\rightarrow \sim 59$ s (2 ranks) $\rightarrow 31.40$ s (3 ranks). With 1 rank, $k = 1, 2, 3$ are computed sequentially; with 3 ranks, all three run in parallel. Inter-rank communication is essentially zero—a single `MPI_Gather` on 12 doubles. Thread count per rank (16/32/64) has negligible impact, confirming the bottleneck is memory bandwidth, not compute.

D. GPU Strong Scaling. Figure 5 shows GPU wall time at $n_Q = 23$. Scaling from ~ 6 s (1 GPU) to 2.10 s (3 GPUs, `COARSEN=1`) tracks close to ideal. The absolute speedup comes from the statevector-reuse optimisation: by branching in-memory copies for each of the four expectation values rather than re-running `PREP+U(t)` four times, three full Trotter sweeps per Krylov vector no longer need to be computed. This is precisely why our OMP backend keeps pace with CUDA-Q despite running on CPU—the algorithmic saving compensates for the hardware gap. `COARSEN=2` gives the best throughput-per-watt; `COARSEN=4` begins to hurt at 1 GPU because per-thread serialisation outweighs the launch-overhead saving.

E. Per-Kernel Profiling (Nsight Systems).

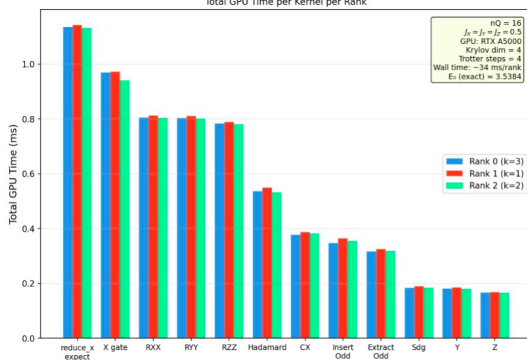


Fig. 6: Nsight Systems per-kernel total GPU time.

Figure 6 reports the total GPU time spent in each kernel, collected via `nsys` profile. The expectation-value reduction `reduce_x_expect` is the single most expensive kernel (~ 1.13 ms per rank), exceeding even the fused two-qubit $R_{XX}/R_{YY}/R_{ZZ}$ rotations. The cost is balanced near-perfectly across the three ranks ($k = 1, 2, 3$), confirming that the round-robin k -assignment produces no load imbalance. The dominance of `reduce_x_expect` over the gate kernels

validates our decision to invest engineering effort in the shared-memory tree reduction with a single per-block `atomicAdd`—a naïve per-thread atomic implementation would have made this kernel an order of magnitude slower and pushed it from ~ 1 ms to dominating the wall time.

V. CONCLUSIONS

In this work, a distributed GPU-accelerated statevector simulator for KQD on the Heisenberg spin chain was presented, validated across CUDA-Q, CUDA/MPI, and OpenMP/MPI backends. Parallel-programming contributions include hand-written CUDA kernels using bit-arithmetic indexing and shared-memory tree reduction; fused 4×4 two-qubit kernels; ancilla-sector extraction halving DRAM traffic; benchmarked thread coarsening; a structurally identical OpenMP/MPI port with kernel-specific `static/dynamic` scheduling; and three-way MPI task parallelism distributing independent Krylov vectors across 3 GPUs with one-shot `MPI_Gather`. Algorithmically, the statevector-reuse optimisation eliminates $3\times$ of the Trotter sweeps required by the IBM circuit-based reference. Together these yield $6.2\times$ speedup over CUDA-Q at 23 qubits, near-ideal strong scaling to 3 GPUs (2.10 s at $n_Q = 23$), and a profiling result confirming the reduction kernel was correctly prioritised.

Future directions: Future work includes deploying the approach on real IBM QPUs (Fez, Kingston, Marrakesh), treating each device as an MPI-equivalent rank. Scaling beyond 30+ qubits via NVLink-enabled statevector sharding and extending the Krylov dimension ($K > 4$) for strongly correlated systems are also promising directions. Further optimisation can be achieved by coalescing the statevector layout so that amplitudes coupled by two-qubit gates are contiguous, improving warp-level memory access for $R_{XX}/R_{YY}/R_{ZZ}$. Finally, stream-parallel evaluation of expectation values—assigning one CUDA stream per Pauli term in H —can enable fully overlapped execution within a single pass.

REFERENCES

- [1] E. N. Epperly, L. Lin, and Y. Nakatsukasa, “A theory of quantum subspace diagonalization,” *SIAM Journal on Matrix Analysis and Applications*, vol. 43, no. 3, pp. 1263–1290, 2022.
- [2] IBM Quantum, “Quantum diagonalization algorithms — Krylov quantum diagonalization,” 2024.
- [3] R. Babbush *et al.*, “Diagonalization of large many-body Hamiltonians on a quantum processor,” *Nature*, 2024.
- [4] IBM, “Qiskit circuit library — HGate,” 2024.
- [5] NVIDIA, “CUDA-Q: The platform for hybrid quantum-classical computing,” 2024.
- [6] SpinQuanta, “Exploring quantum interference: Key concepts explained,” 2024.
- [7] M. Smelyanskiy, N. P. D. Sawaya, and A. Aspuru-Guzik, “qHiPSTER: The quantum high performance software testing environment,” 2016.
- [8] NVIDIA Corporation, “Accelerating quantum circuit simulation with NVIDIA cuStateVec,” 2022.
- [9] M. Suzuki, “General theory of fractal path integrals with applications to many-body theories and statistical physics,” *Journal of Mathematical Physics*, vol. 32, no. 2, pp. 400–407, 1991.